Onboarding Accelerator — Microsoft Defender for Identity

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Onboarding Accelerator: Microsoft Defender for Identity 5

What you get . . . 5

What this delivers . . . 5

The standard deployment model . . . 5

Boundary . . . 5

Effort . . . 6

1. Solution Overview — Onboarding Accelerator: Microsoft Defender for Identity 6

What Microsoft Defender for Identity is . . . 6

Why this matters . . . 6

How it works . . . 6

What you can control . . . 7

Limitations (be honest with buyers) . . . 7

Connections and authentication used . . . 7

The standard deployment model . . . 8

2. Requirements Guide — Onboarding Accelerator: Microsoft Defender for Identity 8

2.1 Defender for Identity activated in the portal . . . 8

2.2 Domain controllers reachable and supported for the sensor . . . 8

2.3 Directory Service Account (DSA) — read-only, gMSA recommended . . . 9

2.4 A pilot domain controller chosen . . . 9

2.5 Sensitive entities identified for tagging . . . 9

2.6 Honeytoken accounts planned . . . 9

2.7 Identity posture (ISPM) scope agreed . . . 9

2.8 Connected response workloads and governance . . . 10

Checklist . . . 10

Global Requirements — Defender (category-level) 10

G1. Microsoft 365 / Entra tenant . . . 10

G2. Defender licensing . . . 10

G3. Administrative roles (least privilege) . . . 11

G4. Onboarding reach to endpoints / identity / workloads . . . 11

G5. Network access to Defender service endpoints . . . 11

G6. A pilot ring . . . 11

G7. Log Analytics workspace (where required) . . . 11

G8. Response governance . . . 11

3. Implementation Guide — Onboarding Accelerator: Microsoft Defender for Identity 12

Naming (set once, reuse verbatim) . . . 12

Step 1 — Assess readiness . . . 12
Step 2 — Activate MDI and prepare the Directory Service Account . . . 12
Step 3 — Install the pilot sensor (pilot DC) . . . 12
Step 4 — Tag sensitive entities (Tier-0) . . . 13
Step 5 — Plant and tag honeytokens . . . 13
Step 6 — Enable and triage identity posture (ISPM) . . . 13
Step 7 — Keep response actions approval-gated . . . 13
Step 8 — Validate detections on the pilot . . . 14
Step 9 — Expand to remaining DCs and AD FS/AD CS . . . 14
5. Testing Guide — Onboarding Accelerator: Microsoft Defender for Identity 14
Test 1 — DSA configured . . . 15
Test 2 — Pilot sensor healthy . . . 15
Test 3 — Sensitive entities tagged . . . 15
Test 4 — Honeytokens tagged . . . 15
Test 5 — Identity signal streaming . . . 15
Test 6 — A detection actually fires . . . 15
Test 7 — ISPM recommendations present . . . 15
Test 8 — Response actions approval-gated . . . 16
6. Troubleshooting Guide — Onboarding Accelerator: Microsoft Defender for Identity 16
DSA not working . . . 16
Sensor unhealthy . . . 16
No identity signal . . . 16
Detection does not fire . . . 17
No ISPM recommendations . . . 17
Honeytoken generating noise . . . 17
Response action acted without approval . . . 17
Script cannot authenticate . . . 17
7. Best Practices — Onboarding Accelerator: Microsoft Defender for Identity 17
Onboard sensors in a ring, never all at once . . . 17
Protect and monitor the sensors . . . 18
Use a gMSA for the Directory Service Account . . . 18
Honeytokens are your highest-signal detection . . . 18
Tag Tier-0 so alerts are weighted correctly . . . 18
Work ISPM as a trend, not a one-off . . . 18
Keep response actions approval-gated until trusted . . . 18
Keep licensing and preview facts current . . . 18

Where to go next 19
8. Deliverables Summary — Onboarding Accelerator: Microsoft Defender for Identity 19
Documentation 19
PowerShell scripts (Microsoft Graph Security / AD read helpers + KQL) 19
What the buyer receives 21
Outcome 21
Appendix A — PowerShell Scripts 22

Executive Summary — Onboarding Accelerator: Microsoft Defender for Identity

White-label · Defender

Most tenants that own Microsoft Defender for Identity (MDI) never finish onboarding it. A sensor is installed on one domain controller, the directory service account is half-configured, honeytokens and sensitive-entity tagging are skipped, and the identity posture (ISPM) recommendations sit un-triaged. The result is an **identity threat detection & response (ITDR)** capability that looks "on" in the portal but sees only a fraction of Active Directory — and nobody has ever confirmed a detection actually fires. This accelerator onboards Defender for Identity **end to end**: it deploys sensors on every domain controller (and AD FS / AD CS where present), configures the read-only **Directory Service Account (DSA)**, plants and tags **honeytokens**, tags **sensitive entities**, turns on and triages **identity posture (ISPM)**, and **validates detections** — piloted first on one domain controller, validated, then expanded to the rest.

This is a complete, **deploy-it-today** package written for an administrator with no prior MDI experience. It onboards a **pilot domain controller first**, keeps any response action **approval-gated**, and never removes an existing control to "test."

What you get

- A plain-language **Solution Overview** of MDI sensors, the DSA, honeytokens, sensitive tags, ISPM posture, and detections.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite.
- An **Implementation Guide** of exact, ordered steps — prepare the DSA, install the pilot sensor, tag entities, plant honeytokens, enable ISPM, validate — pilot-first.
- Five **PowerShell scripts** (Microsoft Graph Security + AD read helpers) plus **KQL advanced-hunting** queries for readiness discovery, sensor/DSA/tagging validation, posture & coverage reporting, a `-WhatIf`-gated honeytoken/sensitive-entity helper, and an identity-detection validation hunt.
- **Testing** and **Troubleshooting** guides for a confident rollout.

What this delivers

A validated, monitored Defender for Identity deployment delivering **ITDR** — sensors healthy on every domain controller, the DSA configured, honeytokens planted and tagged, sensitive entities marked, ISPM recommendations triaged, and detections proven to fire — onboarded in a ring and expanded with confidence, not a half-configured console.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — prepare the DSA and pick a pilot DC, install and health-check one sensor, tag entities and plant honeytokens, validate a detection fires, then expand sensor-by-sensor.

Boundary

This solution **onboards the Defender for Identity pillar itself**. It is distinct from the unified Defender XDR onboarding accelerator (which brings all four pillars into `security.microsoft.com`) and from the SOC/Sentinel integration solutions — cross-reference those, do not duplicate them. Incident-response decisions, automated-response guardrails, and licensing determinations remain the customer's security, operations, and licensing functions; this guide provides the technical configuration.

Effort

A competent administrator can complete the pilot-DC sensor install and detection validation in a focused engagement, then expand across the remaining domain controllers on a controlled change cadence. Sensor rollout scales with domain-controller count and change-window availability.

1. Solution Overview — Onboarding Accelerator: Microsoft Defender for Identity

What Microsoft Defender for Identity is

Microsoft Defender for Identity (MDI) is the **identity threat detection & response (ITDR)** pillar of Microsoft Defender. It detects attacks against **on-premises Active Directory** — and the identity infrastructure around it (AD FS, AD CS) — by installing a lightweight **sensor** directly on domain controllers and federation/certificate servers. The sensor inspects local traffic and events, sends signal to the cloud service, and surfaces alerts in the unified Defender portal at `security.microsoft.com` under **Identities**. MDI catches the identity phase of an intrusion: reconnaissance (LDAP/SMB enumeration), credential theft (Kerberoasting, DCSync, Pass-the-Hash/Ticket), lateral movement, and domain-dominance techniques (Golden Ticket, skeleton key).

Because the sensor runs *on* the domain controller, there is **no agentless option** for on-prem AD — onboarding MDI is fundamentally a **sensor-deployment** exercise, done in a change window, one server at a time.

Why this matters

Almost every attack that reaches an enterprise passes through Active Directory. Attackers who land on one endpoint pivot to AD to escalate, and traditional endpoint or SIEM tooling barely sees that identity movement. MDI is designed exactly for it — but only if it is **fully onboarded**. A tenant with one sensor on one domain controller has blind spots on every other DC; a tenant with no **honeytokens** loses the highest-signal, lowest-noise detections (any touch of a decoy account is suspicious by definition); a tenant that never triages **ISPM** leaves exploitable posture gaps (unconstrained delegation, legacy protocols, weak certificate templates) in place. This accelerator gets the whole pillar onboarded and *validated* — proven that a detection actually fires — rather than assumed-working.

How it works

1. **Directory Service Account (DSA)** — a read-only account (gMSA recommended) is prepared so sensors can query AD for extra context. It is configured under **Settings** → **Identities** → **Directory Service accounts** before sensors are deployed.

2. **Sensors** are installed on **domain controllers** first (then AD FS / AD CS servers). Each sensor is registered with an **access key** from the portal, starts inspecting local traffic, and reports **Running / Healthy**.

3. **Sensitive entities** (Domain Admins, Tier-0 servers, high-value accounts) are **tagged** so MDI weights alerts on them correctly.

4. **Honeytokens** — decoy user/computer accounts that no legitimate process ever touches — are planted and tagged as honeytokens; any authentication or query against them raises a high-confidence alert.

5. **Identity posture (ISPM)** recommendations are enabled and triaged — MDI reports exploitable AD/identity misconfigurations (unconstrained delegation, printer spooler on DCs, legacy protocols, risky certificate templates) that reduce attack surface when fixed.

6. **Detections are validated** — a benign, controlled test (a Microsoft-documented safe simulation or a honeytoken touch) confirms an alert fires in the portal and in advanced hunting.

7. **Response actions** (disable user, force password reset via the connected identity workloads) stay **approval-gated** during the pilot.

What you can control

• **Sensor coverage** — which domain controllers and AD FS/AD CS servers carry a sensor, and the order they onboard (pilot DC first).

• **Directory Service Account** — a read-only gMSA (recommended) versus a standard account.

• **Sensitive-entity tagging** — which accounts, groups, and servers are marked high-value.

• **Honeytokens** — the decoy accounts you plant and tag as tripwires.

• **ISPM triage** — which posture recommendations you remediate first.

• **Response posture** — approval-gated during pilot; automation decisions stay with the SOC.

Limitations (be honest with buyers)

• MDI is **license-gated** (Defender for Identity, commonly via M365 E5 / E5 Security or a standalone plan). Onboarding without the plan fails — *verify current availability in your tenant.*

• **Sensors run on domain controllers** and need resources, network reach to the MDI service, and a **change window**; they are not a portal-only toggle.

• MDI protects **on-premises AD**; cloud-only (Entra ID) identity risk is covered by Entra ID Protection and adjacent solutions, not MDI.

• **Detections, tables, and ISPM** populate over time — allow warm-up after each sensor comes online before judging coverage.

• MDI improves identity detection; it does not replace AD hardening, patching, tiered administration, or the SOC's incident-response decisions.

Connections and authentication used

• The **unified Defender portal** (`security.microsoft.com` → **Settings** → **Identities**) for activation, DSA, sensors, tags, honeytokens, and ISPM.

• **Microsoft Graph Security** and the **Microsoft 365 Defender** APIs (via the `Microsoft.Graph` PowerShell modules) for read-only discovery, validation, reporting, and advanced-hunting (`IdentityInfo`, `IdentityLogonEvents`, `IdentityDirectoryEvents`, `IdentityQueryEvents`, `AlertInfo`).

• **On-premises Active Directory** read helpers (`Get-ADDomainController`, `Get-ADUser`) for local sensor-candidate and honeytoken discovery.

• Least-privilege roles: **Security Administrator** for configuration, **Security Reader** for read-only work, and **Global Administrator** only for one-time first-service activation.

The standard deployment model

> ***Assess readiness (license, roles, DCs, DSA) → configure the DSA and portal settings → install a pilot-DC sensor and plant honeytokens → validate sensor health and that a detection fires → expand to the remaining DCs and AD FS/AD CS.*** *Repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand****, with response actions kept approval-gated until trusted.*

2. Requirements Guide — Onboarding Accelerator: Microsoft Defender for Identity

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant, licensing, admin roles, onboarding reach, network access, pilot ring, workspace, response governance) are in `global-requirements-defender.md` — do not repeat them; the items below are **specific to onboarding Defender for Identity**. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

2.1 Defender for Identity activated in the portal

Description: MDI initialized for the tenant under `security.microsoft.com` → **Settings** → **Identities**. **Why:** The workspace, sensor download, and access key only appear after first-time activation. **Verify:** Sign in as a Security Administrator and confirm **Settings** → **Identities** loads with a workspace name. Run `MDIONB-Discovery.ps1`. **Implement-if-missing:** Complete the one-time MDI activation (Global Administrator required for first-service activation only).

2.2 Domain controllers reachable and supported for the sensor

Description: Domain controllers (and AD FS / AD CS servers where present) that meet the current MDI sensor OS/resource requirements and can reach the MDI service. **Why:** The sensor runs *on* each DC — there is no agentless AD option; unsupported OS or blocked reach means no coverage. **Verify:** `MDIONB-Discovery.ps1` lists every DC (`Get-ADDomainController`), its OS, and reachability hints. Confirm outbound HTTPS to the MDI service URLs. *Verify current sensor OS support and endpoint list in your tenant.* **Implement-if-missing:** Bring the DC OS to a supported level, confirm resources (CPU/RAM/disk), and schedule a change window per DC.

2.3 Directory Service Account (DSA) — read-only, gMSA recommended

Description: A read-only **Directory Service Account** the sensors use to query AD for extra context. Current guidance recommends a **group Managed Service Account (gMSA)**. **Why:** Without the DSA, several detections and enrichment are limited; a gMSA avoids a static password. **Verify:** Confirm the gMSA `MDI Directory Service Account – Tenant – Prod` (or a standard read-only account) is created and installable on the sensor hosts. `MDIONB-Discovery.ps1` reports DSA readiness. *Verify current MDI DSA requirements.* **Implement-if-missing:** Create the gMSA `MDI Directory Service Account – Tenant – Prod`, grant the sensor hosts permission to retrieve its password, and register it under **Settings** → **Identities** → **Directory Service accounts**.

2.4 A pilot domain controller chosen

Description: One domain controller selected as `MDI Sensor Host – Pilot DC – Pilot` to carry the first sensor. **Why:** MDI onboarding is validated on one DC before the rest (per `global-requirements-defender.md` G6); a bad rollout on every DC at once is avoidable. **Verify:** The pilot DC is named and change-approved; `MDIONB-Discovery.ps1` flags it. **Implement-if-missing:** Pick a healthy, representative DC and reserve a change window before installing the pilot sensor.

2.5 Sensitive entities identified for tagging

Description: The high-value accounts, groups, and servers (Tier-0 — Domain Admins, Enterprise Admins, DCs, ADFS/ADCS, PKI, backup, break-glass) to be **tagged sensitive** in MDI. **Why:** MDI weights alerts on tagged sensitive entities more strongly; untagged Tier-0 dilutes signal. **Verify:** A Tier-0 inventory exists; `MDIONB-Validate.ps1` reports which entities are already tagged. **Implement-if-missing:** Build the Tier-0 list and tag under **Settings** → **Identities** → **Entity tags** → **Sensitive** (or use `MDIONB-Honeytoken.ps1 -Mode SensitiveTag` in preview).

2.6 Honeytoken accounts planned

Description: Decoy user/computer accounts — `MDI Honeytoken – Decoy Identity – Pilot` — that no legitimate process ever authenticates or queries. **Why:** Honeytokens are the highest-confidence, lowest-noise MDI detection: any touch is suspicious by definition. **Verify:** The decoy accounts exist in AD and are unused; `MDIONB-Validate.ps1` confirms they are tagged as honeytokens.
Implement-if-missing: Create idle decoy accounts with believable names, ensure nothing uses them, then tag them under **Settings** → **Identities** → **Entity tags** → **Honeytoken**.

2.7 Identity posture (ISPM) scope agreed

Description: Agreement to enable and triage **identity security posture management (ISPM)** recommendations surfaced by MDI into Microsoft Secure Score. **Why:** ISPM finds exploitable AD/identity misconfigurations (unconstrained delegation, spooler on DCs, legacy protocols, risky certificate templates); ignoring them leaves attack surface open. **Verify:** `security.microsoft.com` → **Secure Score** → **Recommended actions** (Identity) shows MDI recommendations after sensors are healthy. `MDIONB-Report.ps1` lists them. *Verify current ISPM availability.* **Implement-if-missing:** Confirm sensors are healthy (ISPM depends on sensor data), then review the Identity recommendations and assign owners.

2.8 Connected response workloads and governance

Description: For MDI response actions (disable user, force password reset), the on-prem/hybrid identity workloads must be connected, and a response-governance position agreed. **Why:** Response actions are disruptive; the organization decides the guardrails (per `global-requirements-defender.md` G8). **Verify:** A documented response policy exists; actions are approval-gated. `MDIONB-Validate.ps1` reports the current action configuration. **Implement-if-missing:** Keep response actions approval-gated at first; *incident-response and automation guardrail decisions are the SOC/security team's — this guide covers the technical configuration.*

Checklist

- MDI activated in the portal — `MDIONB-Discovery.ps1`
- Domain controllers supported/reachable for the sensor
- DSA (gMSA) `MDI Directory Service Account – Tenant – Prod` prepared
- Pilot DC `MDI Sensor Host – Pilot DC – Pilot` chosen + change window
- Sensitive entities (Tier-0) identified for tagging
- Honeytoken accounts `MDI Honeytoken – Decoy Identity – Pilot` planned
- ISPM scope agreed — `MDIONB-Report.ps1`
- Response governance (approval-gated) documented

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify:

Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Onboarding Accelerator: Microsoft Defender for Identity

Follow these steps **in order**. Most configuration is in the **unified Defender portal** (`https://security.microsoft.com` → **Settings** → **Identities**); sensor install runs on the domain controllers themselves. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model:*** *Prepare the DSA, install the sensor on **one pilot domain controller** first, tag entities and plant honeytokens, **validate that a detection actually fires**, and only then expand to the remaining DCs and AD FS/AD CS. Keep **response actions** approval-gated throughout. Never remove an existing AD control to "test."*

Naming (set once, reuse verbatim)

- Directory Service Account (gMSA): `MDI Directory Service Account – Tenant – Prod`
- Pilot sensor host: `MDI Sensor Host – Pilot DC – Pilot`
- Honeytoken decoy account(s): `MDI Honeytoken – Decoy Identity – Pilot`
- Sensitive-entity tag scope: `MDI Sensitive Entities – Tier0 – Prod`
- Advanced-hunting saved query: `MDI Detection Validation – Tenant – Validate`

Step 1 — Assess readiness

1. Sign in to `security.microsoft.com` as a **Security Administrator**.
2. Run `MDIONB-Discovery.ps1` to capture the current state: MDI activation/workspace, licensed plan hints, admin roles, every domain controller and its OS, DSA readiness, and existing sensor status.
3. Choose the pilot DC and record it as `MDI Sensor Host – Pilot DC – Pilot`. Reserve a change window.

Step 2 — Activate MDI and prepare the Directory Service Account

1. `security.microsoft.com` → **Settings** → **Identities**. If MDI is not initialized, complete the one-time activation and note the **workspace** name.
2. Create the read-only gMSA `MDI Directory Service Account – Tenant – Prod` in AD, and grant the sensor host computer accounts permission to retrieve its password.
3. **Settings** → **Identities** → **Directory Service accounts** → **Add credentials** — register `MDI Directory Service Account – Tenant – Prod` (gMSA). *Verify current MDI DSA requirements.*
4. Run `MDIONB-Validate.ps1 -Check DSA` to confirm the DSA is registered.

Step 3 — Install the pilot sensor (pilot DC)

1. `security.microsoft.com` → **Settings** → **Identities** → **Sensors** → **Add sensor** — download the sensor package and copy the **access key**.

2. During the approved change window, copy the package to `MDI Sensor Host – Pilot DC – Pilot`, run the installer, and supply the **access key** when prompted.

3. Confirm outbound HTTPS from the DC to the MDI service URLs (per `global-requirements-defender.md` G5). *Verify current endpoint list.*

4. **Settings** → **Identities** → **Sensors** — confirm `MDI Sensor Host – Pilot DC – Pilot` shows **Running / Healthy**.

5. Run `MDIONB-Validate.ps1 -Check Sensor` to confirm the pilot sensor is healthy and identity signal is flowing.

Step 4 — Tag sensitive entities (Tier-0)

1. `security.microsoft.com` → **Settings** → **Identities** → **Entity tags** → **Sensitive**.

2. Add the Tier-0 accounts, groups, and servers as `MDI Sensitive Entities – Tier0 – Prod` (Domain/Enterprise Admins, DCs, AD FS/AD CS, PKI, backup, break-glass).

3. Preview the change first with `MDIONB-Honeytoken.ps1 -Mode SensitiveTag` (safe preview by default); apply only with `-Apply`.

4. Run `MDIONB-Validate.ps1 -Check Sensitive` to confirm the entities are tagged.

Step 5 — Plant and tag honeytokens

1. In AD, create the idle decoy account(s) `MDI Honeytoken – Decoy Identity – Pilot` with believable names; confirm **nothing legitimate uses them** (no logon scripts, services, or scheduled tasks).

2. `security.microsoft.com` → **Settings** → **Identities** → **Entity tags** → **Honeytoken** — add the decoy account(s).

3. Preview with `MDIONB-Honeytoken.ps1 -Mode Honeytoken` (safe preview); apply with `-Apply`.

4. Run `MDIONB-Validate.ps1 -Check Honeytoken` to confirm the accounts are tagged as honeytokens.

Step 6 — Enable and triage identity posture (ISPM)

1. After the pilot sensor is healthy, `security.microsoft.com` → **Secure Score** → **Recommended actions**, filter to **Identity** — MDI ISPM recommendations appear (unconstrained delegation, spooler on DCs, legacy protocols, risky certificate templates). *Verify current ISPM availability.*

2. Record the starting posture with `MDIONB-Report.ps1 -SaveBaseline` and assign owners to the top recommendations.

3. *Which posture items are "must-fix" and the remediation change-approval are the security/AD team's decisions — this guide surfaces the recommendations and the technical fix path.*

Step 7 — Keep response actions approval-gated

1. `security.microsoft.com` → **Settings** → **Identities** → **Response actions** (where available) — confirm response actions (disable user, force password reset) require approval, not automatic execution, during the pilot. *Verify current response-action availability in your tenant.*

2. Record the current configuration with `MDIONB-Report.ps1`.

3. *The decision to enable any automatic response is the SOC/security team's — this guide leaves it approval-gated.*

Step 8 — Validate detections on the pilot

1. Run `MDIONB-Hunt.ps1` to execute `MDI Detection Validation – Tenant – Validate` — it confirms the identity tables (`IdentityInfo`, `IdentityLogonEvents`, `IdentityDirectoryEvents`, `IdentityQueryEvents`) are streaming from the pilot sensor.

2. Trigger a **benign, controlled** detection: authenticate against or query the honeytoken `MDI Honeytoken – Decoy Identity – Pilot`, or run a Microsoft-documented safe simulation. Confirm an alert appears in `security.microsoft.com` → **Incidents & alerts**.

3. Complete the **Testing Guide** before expanding.

Step 9 — Expand to remaining DCs and AD FS/AD CS

1. **Sensors:** install the sensor on each remaining domain controller in change windows until all show **Running / Healthy**; then AD FS / AD CS servers.

2. **Tags/honeytokens:** extend sensitive tagging and add honeytokens across additional forests/domains as needed.

3. **ISPM:** re-run `MDIONB-Report.ps1 -SaveBaseline` after full sensor coverage — posture recommendations get more complete as more DCs report.

4. **Response:** only after coverage and detections are trusted, review response-action automation with the SOC (still their decision).

5. Track sensor-health and posture drift each cycle with `MDIONB-Report.ps1`.

Order of operations

```
Assess → activate MDI + DSA → install pilot-DC sensor → tag sensitive
entities → plant honeytokens → enable/triage ISPM → keep response
approval-gated → validate detections → expand to all DCs + AD FS/AD CS
```

5. Testing Guide — Onboarding Accelerator: Microsoft Defender for Identity

Run these checks after the pilot onboarding (Steps 2–8) and again after each expansion wave. Each test has a **pass/fail gate**; do not expand to the remaining domain controllers until the pilot passes. Objects are referenced by the exact names from the Implementation Guide.

Test 1 — DSA configured

Procedure: Run `MDIONB-Validate.ps1 -Check DSA`. In `security.microsoft.com` → **Settings** → **Identities** → **Directory Service accounts**, confirm `MDI Directory Service Account – Tenant – Prod` is present. **Pass:** The DSA (gMSA) is registered and the sensor can retrieve its password. **Fail:** DSA missing or password-retrieval fails — see Troubleshooting *DSA not working*.

Test 2 — Pilot sensor healthy

Procedure: Run `MDIONB-Validate.ps1 -Check Sensor`. In **Settings** → **Identities** → **Sensors**, confirm `MDI Sensor Host – Pilot DC – Pilot` is **Running / Healthy**. **Pass:** The pilot sensor is healthy and reporting; no health alerts against it. **Fail:** Sensor "Unhealthy / Not configured / Stopped" — see Troubleshooting *Sensor unhealthy*.

Test 3 — Sensitive entities tagged

Procedure: Run `MDIONB-Validate.ps1 -Check Sensitive`. In **Settings** → **Identities** → **Entity tags** → **Sensitive**, confirm the `MDI Sensitive Entities – Tier0 – Prod` accounts/groups/servers are tagged. **Pass:** All planned Tier-0 entities are tagged sensitive. **Fail:** Tier-0 entities missing — re-apply per Implementation Step 4.

Test 4 — Honeytokens tagged

Procedure: Run `MDIONB-Validate.ps1 -Check Honeytoken`. In **Settings** → **Identities** → **Entity tags** → **Honeytoken**, confirm `MDI Honeytoken – Decoy Identity – Pilot` is tagged. **Pass:** The decoy account(s) are tagged as honeytokens and remain unused. **Fail:** Not tagged, or the account shows real usage — re-apply per Implementation Step 5 and confirm nothing legitimate uses the account.

Test 5 — Identity signal streaming

Procedure: Run `MDIONB-Hunt.ps1` to execute `MDI Detection Validation – Tenant – Validate`. Confirm `IdentityInfo`, `IdentityLogonEvents`, `IdentityDirectoryEvents`, and `IdentityQueryEvents` return recent rows. **Pass:** All four identity tables stream from the pilot sensor. **Fail:** A table is empty — see Troubleshooting *No identity signal*.

Test 6 — A detection actually fires

Procedure: Trigger a benign, controlled detection — authenticate against or query the honeytoken `MDI Honeytoken – Decoy Identity – Pilot`, or run a Microsoft-documented safe simulation. In `security.microsoft.com` → **Incidents & alerts**, confirm the resulting alert appears. **Pass:** The controlled test produces a visible MDI alert within the expected window. **Fail:** No alert — see Troubleshooting *Detection does not fire*.

Test 7 — ISPM recommendations present

Procedure: Run `MDIONB-Report.ps1 -SaveBaseline`. In **Secure Score** → **Recommended actions** (Identity), confirm MDI ISPM recommendations are listed. **Pass:** ISPM recommendations appear and a posture baseline is recorded. **Fail:** No recommendations — allow sensor warm-up time, confirm sensor health (Test 2), then re-run; see Troubleshooting *No ISPM recommendations*.

Test 8 — Response actions approval-gated

Procedure: Run `MDIONB-Report.ps1` and confirm response actions are approval-gated (not automatic) per Implementation Step 7. **Pass:** No response action auto-executes during the pilot; actions require approval. **Fail:** Automation is higher than agreed — reset per Implementation Step 7 and confirm with the SOC.

Exit gate

All eight tests pass on the pilot DC **before** expanding to the remaining domain controllers (Implementation Step 9). Re-run Tests 2, 5, 6, and 7 after each sensor-expansion wave and compare against the saved posture baseline.

6. Troubleshooting Guide — Onboarding Accelerator: Microsoft Defender for Identity

Common field issues, in **symptom** → **cause** → **fix** form. Run `MDIONB-Discovery.ps1` and `MDIONB-Validate.ps1` first — most causes surface there.

DSA not working

Symptom: Test 1 fails — `MDI Directory Service Account – Tenant – Prod` is missing, or the sensor cannot use it and enrichment detections are limited. **Cause:** The gMSA is not registered under **Directory Service accounts**, the sensor host computer accounts lack permission to retrieve the gMSA password, or a standard account's password is wrong. **Fix:** Confirm the DSA is registered under **Settings** → **Identities** → **Directory Service accounts**; grant the sensor hosts permission to retrieve the gMSA password; re-enter credentials for a standard account. *Verify current MDI DSA requirements.* Re-run `MDIONB-Validate.ps1 -Check DSA`.

Sensor unhealthy

Symptom: `MDI Sensor Host – Pilot DC – Pilot` shows **Unhealthy / Not configured / Stopped** in **Settings** → **Identities** → **Sensors**. **Cause:** Blocked outbound reach to the MDI service, insufficient DC resources, an unsupported sensor OS, a wrong/expired access key, or a proxy in the way. **Fix:** Confirm the DC can reach the MDI service URLs (per `global-requirements-defender.md` G5); confirm DC resources and a supported OS version (*verify current sensor OS support*); reinstall with a fresh access key from **Add sensor** if needed; configure the sensor proxy if one is required. Re-run `MDIONB-Validate.ps1 -Check Sensor`.

No identity signal

Symptom: Test 5 fails — one or more of `IdentityInfo`, `IdentityLogonEvents`, `IdentityDirectoryEvents`, `IdentityQueryEvents` return no rows. **Cause:** The sensor is unhealthy or too newly installed, or not enough traffic has passed the DC yet for tables to populate. **Fix:** Confirm the pilot sensor is healthy first (Test 2). Allow warm-up time — identity tables populate over hours. Generate benign authentication traffic through the DC and re-run `MDIONB-Hunt.ps1`.

Detection does not fire

Symptom: Test 6 fails — the controlled honeytoken touch or safe simulation produces no alert. **Cause:** The honeytoken is not actually tagged, the sensor is unhealthy, the test did not reach a sensored DC, or not enough time has passed for the alert. **Fix:** Confirm the honeytoken is tagged (Test 4) and the sensor healthy (Test 2); ensure the authentication/query targets a DC that has a sensor; allow time for the alert to surface in **Incidents & alerts**. Re-test with `MDIONB-Hunt.ps1` to confirm the query event is recorded.

No ISPM recommendations

Symptom: Test 7 fails — no MDI Identity recommendations appear in **Secure Score** → **Recommended actions**. **Cause:** Sensors are too newly onboarded for ISPM to compute, sensor coverage is too thin, or ISPM is not yet available in the tenant. **Fix:** Confirm sensor health (Test 2) and allow warm-up; expand sensor coverage so more DCs report; *verify current ISPM availability in your tenant.* Re-run `MDIONB-Report.ps1 -SaveBaseline`.

Honeytoken generating noise

Symptom: The honeytoken account fires alerts from legitimate activity. **Cause:** A real process (logon script, service, scheduled task, or a person) is using the decoy account, so it is no longer idle. **Fix:** Identify and remove the legitimate use (`MDIONB-Hunt.ps1` shows what touched it), or retire the account and create a genuinely idle replacement named `MDI Honeytoken – Decoy Identity – Pilot`. Re-tag and re-validate (Test 4).

Response action acted without approval

Symptom: A user was disabled or a password reset forced automatically during the pilot. **Cause:** A response action was configured for automatic execution rather than approval-gated. **Fix:** Reset the action to approval-gated (Implementation Step 7), confirm with the SOC, and record the configuration with `MDIONB-Report.ps1`. *The automation-guardrail decision is the security team's.*

Script cannot authenticate

Symptom: `MDIONB-*.ps1` fails on connect or returns access-denied. **Cause:** The Microsoft Graph modules are missing, consent to the required read scopes was not granted, the account lacks Security Reader, or the AD read helpers are run off a machine without the AD module. **Fix:** Let the script install the modules (it checks on run); sign in with `Connect-MgGraph` and consent to the read-only scopes; confirm Security Reader (or higher); run the AD-inventory portion from a domain-joined host with `RSAT-AD-PowerShell`. Re-run the script.

7. Best Practices — Onboarding Accelerator: Microsoft Defender for Identity

Field-proven guidance for keeping the Defender for Identity deployment healthy, high-signal, and low-risk.

Onboard sensors in a ring, never all at once

• Install the sensor on **one pilot domain controller** (`MDI Sensor Host – Pilot DC – Pilot`) first, health-check it, validate a detection, then expand DC-by-DC in change windows. A bad rollout on every DC simultaneously is avoidable.

• Cover **every** domain controller eventually — a single un-sensored DC is a blind spot attackers can use.

Protect and monitor the sensors

• Domain-controller sensors are security-critical infrastructure. Keep them healthy, patched, and monitored; a silent sensor blinds MDI. Review sensor health on a cadence with `MDIONB-Report.ps1`.

• Extend sensors to **AD FS and AD CS** servers — federation and certificate services are prime attacker targets and MDI has detections specific to them.

Use a gMSA for the Directory Service Account

• Prefer a **group Managed Service Account** for `MDI Directory Service Account – Tenant – Prod` so there is no static password to rotate or leak. Grant only the read access MDI needs. *Verify current MDI DSA requirements.*

Honeytokens are your highest-signal detection

• Plant decoy accounts (`MDI Honeytoken – Decoy Identity – Pilot`) that **nothing legitimate ever touches**, tag them, and treat any hit as high-confidence. Give them believable names and place them where an attacker enumerating AD would find them.

• Keep them genuinely idle — a honeytoken used by a real process becomes noise.

Tag Tier-0 so alerts are weighted correctly

• Tag Domain/Enterprise Admins, DCs, AD FS/AD CS, PKI, backup, and break-glass as sensitive (`MDI Sensitive Entities – Tier0 – Prod`). MDI weights alerts on tagged entities; untagged Tier-0 dilutes the signal that matters most.

Work ISPM as a trend, not a one-off

• Triage the **identity posture** recommendations by exploitability (unconstrained delegation, spooler on DCs, legacy protocols, risky certificate templates) and track the delta each cycle with `MDIONB-Report.ps1 -SaveBaseline`. Re-baseline after full sensor coverage.

Keep response actions approval-gated until trusted

• Leave MDI response actions (disable user, force password reset) approval-gated during the pilot and early expansion. Moving toward automatic response is the SOC/security team's decision, made only after coverage and detections are proven.

Keep licensing and preview facts current

• MDI and some features are **license-gated** and evolve. Before relying on a capability, **verify current availability and plan requirements in your tenant** — do not assume a feature is included.

Where to go next

Once MDI is onboarded and validated, natural next steps in the catalog:

• The **Microsoft Defender XDR — Unified Onboarding** accelerator to bring MDI together with Endpoint, Office 365, and Cloud Apps into one correlated portal.

• The **Onboarding Accelerator — Microsoft Defender for Endpoint** to add the endpoint pillar an identity attack usually pivots through.

• The **SOC onboarding — Sentinel + Defender XDR** and **advanced KQL threat-hunting** solutions to hunt identity attacks in a SIEM.

• **Entra ID governance / Conditional Access / PIM** solutions to harden the cloud identity layer alongside on-prem AD.

8. Deliverables Summary — Onboarding Accelerator: Microsoft Defender for Identity

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What MDI is — sensors, DSA, honeytokens, sensitive tags, ISPM posture, detections; benefits, limitations
2. Requirements Guide	Solution-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps — DSA, pilot-DC sensor, sensitive tagging, honeytokens, ISPM, detection validation, expand
5. Testing Guide	Eight validation tests with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations and where to go next

PowerShell scripts (Microsoft Graph Security / AD read helpers + KQL)

Script	What it does			
`MDIONB-Discovery.ps1`	Read-only readiness — MDI activation/workspace, plan hints, admin roles, every domain controller and OS, DSA readiness, existing sensor status			
`MDIONB-Validate.ps1`	Targeted validation (`-Check DSA\	Sensor\	Sensitive\	Honeytoken`) — confirms the DSA, sensor health, sensitive-entity tags, and honeytoken tags
`MDIONB-Report.ps1`	Sensor-health, ISPM posture, and coverage report with top Identity recommendations and drift versus a saved baseline; reports response-action posture			
`MDIONB-Honeytoken.ps1`	`-WhatIf`-gated helper to tag sensitive entities (`-Mode SensitiveTag`) and honeytokens (`-Mode Honeytoken`) — safe preview by default			

Script	What it does			
`MDIONB-Hunt.ps1`	Runs the KQL advanced-hunting query `MDI Detection Validation` via Graph Security to confirm the identity tables stream and a test detection is visible (read-only)			

What the buyer receives

- This solution as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** (with embedded KQL) in a single ZIP.
- A repeatable process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-DC sensor onboarding with response actions kept approval-gated until trusted.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A validated, monitored Defender for Identity deployment delivering ITDR — sensors healthy on every domain controller, the DSA configured, honeytokens planted and tagged, sensitive entities marked, ISPM recommendations triaged, and a detection proven to fire — onboarded on a pilot DC before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

MDIONB-Discovery.ps1

Purpose: Read-only discovery for the "Onboarding Accelerator — Microsoft Defender for Identity" solution. Reports the signed-in context, licensed Defender-for-Identity plan hints, the admin roles held, every domain controller (and AD FS/AD CS candidate) with its OS, and Directory Service Account readiness. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDIONB-Discovery.ps1 Requires the Microsoft.Graph modules (Microsoft.Graph.Authentication, Microsoft.Graph.Identity.DirectoryManagement). The AD inventory portion uses the RSAT ActiveDirectory module and should run from a domain-joined host. The script connects read-only with Connect-MgGraph and prompts for consent to read scopes. EXPECTED RESULTS A readiness picture: tenant/context, plan hints that indicate MDI entitlement, admin roles, the domain-controller inventory to sensor, and DSA notes — the state to confirm before you begin onboarding. Written to a JSON snapshot. .NOTES White-label. Read-only. Reports on configuration/identifiers only; makes no changes. Licensing/plan facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDIONB-discovery.json")

How to run: powershell -ExecutionPolicy Bypass -File .\MDIONB-Discovery.ps1 Requires the Microsoft.Graph modules (Microsoft.Graph.Authentication, Microsoft.Graph.Identity.DirectoryManagement). The AD inventory portion uses the RSAT ActiveDirectory module and should run from a domain-joined host. The script connects read-only with Connect-MgGraph and prompts for consent to read scopes. EXPECTED RESULTS A readiness picture: tenant/context, plan hints that indicate MDI entitlement, admin roles, the domain-controller inventory to sensor, and DSA notes — the state to confirm before you begin onboarding. Written to a JSON snapshot. .NOTES White-label. Read-only. Reports on configuration/identifiers only; makes no changes. Licensing/plan facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDIONB-discovery.json")

MDIONB-Honeytoken.ps1

Purpose: Change helper for the "Onboarding Accelerator — Microsoft Defender for Identity" solution. Adds the planned entities to the MDI Sensitive tag ('MDI Sensitive Entities - Tier0 - Prod') or the Honeytoken tag ('MDI Honeytoken - Decoy Identity - Pilot'). This is the only script that can change configuration, and it defaults to a safe preview. HOW TO RUN Preview (default — shows what WOULD be tagged, changes nothing): powershell -ExecutionPolicy Bypass -File .\MDIONB-Honeytoken.ps1 -Mode Honeytoken -InputCsv .\honeytokens.csv Apply (explicit — required to make the change): powershell -ExecutionPolicy Bypass -File .\MDIONB-Honeytoken.ps1 -Mode Honeytoken -InputCsv .\honeytokens.csv -Apply -Mode accepts: SensitiveTag | Honeytoken. The CSV has one column: samAccountName (or the entity object id). Requires Microsoft.Graph with the MDI tag write scope. EXPECTED RESULTS Preview: prints each resolved entity and the tag it WOULD receive. Apply: applies the tag and confirms. Plant decoys that nothing legitimate uses; validate with MDIONB-Validate.ps1 -Check Honeytoken. .NOTES White-label. DEFAULTS TO -WhatIf/preview. Honeytokens must be genuinely idle accounts. Where the tag API is unavailable in your tenant, the script prints the exact portal path. > [CmdletBinding()] param([Parameter(Mandatory)][ValidateSet('SensitiveTag','Honeytoken')] [string]$Mode, [Parameter(Mandatory)] [string]$InputCsv, [switch]$Apply)

How to run: Preview (default — shows what WOULD be tagged, changes nothing): powershell -ExecutionPolicy Bypass -File .\MDIONB-Honeytoken.ps1 -Mode Honeytoken -InputCsv .\honeytokens.csv Apply (explicit — required to make the change): powershell -ExecutionPolicy Bypass -File .\MDIONB-Honeytoken.ps1 -Mode Honeytoken -InputCsv .\honeytokens.csv -Apply -Mode accepts: SensitiveTag | Honeytoken. The CSV has one column: samAccountName (or the entity object id). Requires Microsoft.Graph with the MDI tag write scope. EXPECTED RESULTS Preview: prints each resolved entity and the tag it WOULD receive. Apply: applies the tag and confirms. Plant decoys that nothing legitimate uses; validate with MDIONB-Validate.ps1 -Check Honeytoken. .NOTES White-label. DEFAULTS TO -WhatIf/preview. Honeytokens must be genuinely idle accounts. Where the tag API is unavailable in your tenant, the script prints the exact portal path. > [CmdletBinding()] param([Parameter(Mandatory)][ValidateSet('SensitiveTag','Honeytoken')] [string]$Mode, [Parameter(Mandatory)] [string]$InputCsv, [switch]$Apply)

MDIONB-Hunt.ps1

Purpose: Read-only advanced-hunting for the "Onboarding Accelerator — Microsoft Defender for Identity" solution. Executes the saved query 'MDI Detection Validation - Tenant - Validate' via Microsoft Graph Security advanced hunting to confirm the identity tables are populated from the pilot sensor, and to show recent activity against the honeytoken account so you can confirm a test detection fired (Testing Guide Tests 5 and 6). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDIONB-Hunt.ps1 -HoneytokenSam 'svc-legacy-bak' Uses Microsoft.Graph.Authentication and the ThreatHunting.Read.All scope. Read-only. EXPECTED RESULTS Row counts per identity table and any recent events touching the honeytoken — proof that the sensor is streaming and that a controlled honeytoken touch produced signal. .NOTES White-label. Read-only (advanced hunting). Embedded KQL below is the delivered query. > [CmdletBinding()] param([string]$HoneytokenSam = 'REPLACE-with-honeytoken-samAccountName', [int]$LookbackDays = 1)

How to run: powershell -ExecutionPolicy Bypass -File .\MDIONB-Hunt.ps1 -HoneytokenSam 'svc-legacy-bak' Uses Microsoft.Graph.Authentication and the ThreatHunting.Read.All scope. Read-only. EXPECTED RESULTS Row counts per identity table and any recent events touching the honeytoken — proof that the sensor is streaming and that a controlled honeytoken touch produced signal. .NOTES White-label. Read-only (advanced hunting). Embedded KQL below is the delivered query. > [CmdletBinding()] param([string]$HoneytokenSam = 'REPLACE-with-honeytoken-samAccountName', [int]$LookbackDays = 1)

MDIONB-Report.ps1

Purpose: Read-only reporting for the "Onboarding Accelerator — Microsoft Defender for Identity" solution. Summarizes sensor health across domain controllers, the MDI/Identity Secure Score recommendations (ISPM), and — with -SaveBaseline — records a posture baseline so later runs show drift. Also reports the response-action posture (should be approval-gated). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDIONB-Report.ps1 -SaveBaseline powershell -ExecutionPolicy Bypass -File .\MDIONB-Report.ps1 # compare to saved baseline Uses Microsoft.Graph.Authentication + Graph Security (Secure Score, secure-score-control-profiles). EXPECTED RESULTS A sensor-health summary, the current Identity Secure Score and top recommendations, and the delta versus the saved baseline. A trend of remediated recommendations = improving posture. .NOTES White-label. Read-only. ISPM/Secure Score facts are indicative — verify current availability. Response actions must remain approval-gated until the SOC decides otherwise. > [CmdletBinding()] param([switch]$SaveBaseline, [string]$BaselinePath = ".\MDIONB-posture-baseline.json")

How to run: powershell -ExecutionPolicy Bypass -File .\MDIONB-Report.ps1 -SaveBaseline powershell -ExecutionPolicy Bypass -File .\MDIONB-Report.ps1 # compare to saved baseline Uses Microsoft.Graph.Authentication + Graph Security (Secure Score, secure-score-control-profiles). EXPECTED RESULTS A sensor-health summary, the current Identity Secure Score and top recommendations, and the delta versus the saved baseline. A trend of remediated recommendations = improving posture. .NOTES White-label. Read-only. ISPM/Secure Score facts are indicative — verify current availability. Response actions must remain approval-gated until the SOC decides otherwise. > [CmdletBinding()] param([switch]$SaveBaseline, [string]$BaselinePath = ".\MDIONB-posture-baseline.json")

MDIONB-Validate.ps1

Purpose: Read-only validation for the "Onboarding Accelerator — Microsoft Defender for Identity" solution. Confirms the piece named by -Check: the Directory Service Account, pilot sensor health, sensitive-entity tagging, or honeytoken tagging. Maps to the Testing Guide gates. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDIONB-Validate.ps1 -Check Sensor -Check accepts: DSA | Sensor | Sensitive | Honeytoken | All (default All). Uses Microsoft.Graph.Authentication and the Microsoft 365 Defender / Graph Security APIs. Reads the MDI security profile; makes no changes. EXPECTED RESULTS PASS/REVIEW lines per check with the object names from the Implementation Guide, so you can confirm each Testing Guide gate before expanding beyond the pilot domain controller. .NOTES White-label. Read-only. Some MDI settings are surfaced through the portal only — where an API read is unavailable the script prints the exact portal path to confirm manually. > [CmdletBinding()] param([ValidateSet('DSA','Sensor','Sensitive','Honeytoken','All')] [string]$Check = 'All')

How to run: powershell -ExecutionPolicy Bypass -File .\MDIONB-Validate.ps1 -Check Sensor -Check accepts: DSA | Sensor | Sensitive | Honeytoken | All (default All). Uses Microsoft.Graph.Authentication and the Microsoft 365 Defender / Graph Security APIs. Reads the MDI security profile; makes no changes. EXPECTED RESULTS PASS/REVIEW lines per check with the object names from the Implementation Guide, so you can confirm each Testing Guide gate before expanding beyond the pilot domain controller. .NOTES White-label. Read-only. Some MDI settings are surfaced through the portal only — where an API read is unavailable the script prints the exact portal path to confirm manually. > [CmdletBinding()] param([ValidateSet('DSA','Sensor','Sensitive','Honeytoken','All')] [string]$Check = 'All')